

Teori & Analisis Algoritma binary tree

a. Pohon Ekspresi & Traversal

Fungsi `_buildTree()` membangun pohon ekspresi dengan membaca token satu per satu dari antrian. Ketika token berupa “(”, program membuat subtree kiri terlebih dahulu, kemudian membaca operator, lalu membuat subtree kanan. Jika token berupa angka, maka angka langsung menjadi leaf node. Setelah subtree selesai, tanda “)” diabaikan. Traversal postorder otomatis menghasilkan postfix notation karena subtree kiri dan kanan diproses terlebih dahulu sebelum operator dicetak. Sedangkan inorder traversal membutuhkan tanda kurung tambahan agar urutan operasi tetap benar. Jika pohon memiliki tinggi h , maka kedalaman maksimum stack rekursi juga sebesar h karena setiap pemanggilan rekursi mengikuti kedalaman node pada pohon.

b. Struktur Heap & Pemetaan Array

Rumus $\text{parent} = (i-1)/2$ dan $\text{child} = 2*i+1$ hanya valid pada complete binary tree karena semua node harus tersusun rapat dari kiri ke kanan. Jika terdapat lubang pada struktur pohon, hubungan indeks parent-child menjadi tidak sesuai. Fungsi `sift-down()` bekerja dengan membandingkan parent terhadap child terbesar lalu melakukan swap jika child lebih besar. Proses ini terus diulang hingga heap order property kembali benar. Jumlah maksimum perbandingan pada satu sift-down sekitar $2 \log n$ karena setiap level heap membutuhkan maksimal dua perbandingan.

c. Heapsort In-Place vs Simple

Simple heapsort biasanya membutuhkan heap tambahan sehingga penggunaan memori lebih besar. Sedangkan in-place heapsort langsung menggunakan array yang sama sehingga kompleksitas ruang tambahan hanya $O(1)$. In-place heapsort juga memiliki cache locality yang lebih baik karena semua operasi dilakukan pada array yang sama tanpa membuat salinan data baru. Hal ini lebih aman untuk sistem dengan RAM terbatas. Meskipun terjadi banyak swap pada array yang sama, kompleksitas waktu tetap $O(n \log n)$ karena setiap sift-down membutuhkan $O(\log n)$ dan dilakukan sebanyak n kali.

d. Batas Teoretis & Decision Tree

Heapsort tidak melanggar batas $\Omega(n \log n)$ karena algoritma ini tetap termasuk comparison sort. Semua keputusan sorting dilakukan berdasarkan perbandingan antar elemen. Pada decision tree seperti decoding Morse Code, null link digunakan untuk mendeteksi token tidak valid. Jika traversal menuju node None, maka urutan simbol dianggap salah atau tidak tersedia pada pohon. Pendekatan rekursif lebih cocok digunakan karena struktur pohon bersifat hierarkis sehingga traversal kiri dan kanan lebih mudah dilakukan secara alami dibanding iteratif.